

UNIVERSIDAD TECNOLÓGICA DE PEREIRA

Laboratorio N.º 1

Comunicación entre procesos mediante sockets TCP

Santiago Guevara Méndez

Sistemas Distribuidos · Prof. Felipe Gutiérrez Isaza · 2026-2

Fecha de entrega: 6 de octubre de 2026

1. Objetivos

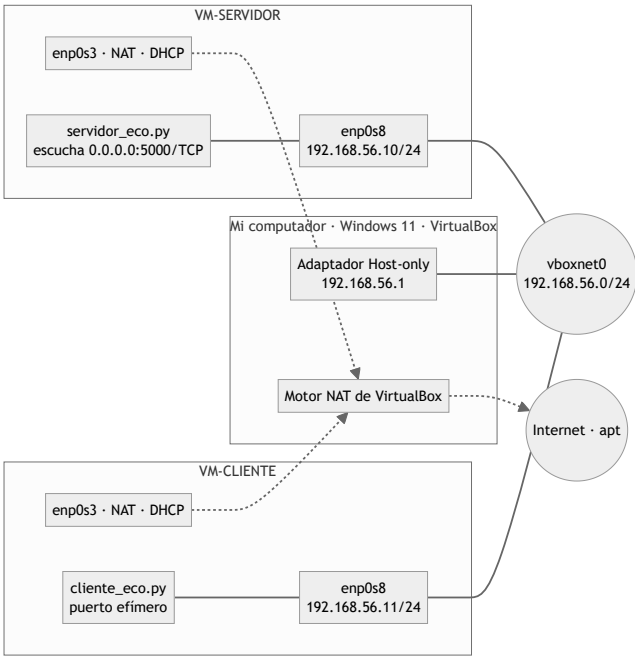
General. Implementar y observar una comunicación cliente/servidor sobre TCP entre dos máquinas virtuales, para comprobar experimentalmente qué garantiza el canal (fiabilidad y orden) y qué no garantiza (fronteras de mensaje, detección inmediata de fallos).

Específicos. (1) Configurar una red Host-only entre dos VM y verificarla por capas (RA1). (2) Implementar en Python los dos extremos de una conexión TCP con la API de sockets (RA2). (3) Explicar el ciclo de vida de la conexión y qué parte ocurre en el núcleo del sistema operativo (RA3). (4) Identificar conexiones por su cuádrupla y justificar los dos sockets del servidor (RA4). (5) Demostrar que TCP entrega un flujo de bytes sin fronteras y diseñar un protocolo con delimitador (RA5, reto R2). (6) Diagnosticar fallos distinguiendo la capa de origen (RA6).

2. Descripción del montaje

Máquina	Rol	Interfaz NAT	Interfaz del laboratorio	Programa
VM-SERVIDOR	servidor	enp0s3 (DHCP)	enp0s8 · 192.168.56.10/24	servidor_eco.py, puerto 5000/TCP
VM-CLIENTE	cliente	enp0s3 (DHCP)	enp0s8 · 192.168.56.11/24	cliente_eco.py, puerto efímero

Las dos VM usan Ubuntu 26.04.1 LTS (kernel 7.0.0-34) y Python 3.14.4. VM-CLIENTE es un clon de VM-SERVIDOR con las MAC reinicializadas y otro nombre de host. Configuré la IP fija de enp0s8 con netplan siguiendo red/DESPLIEGUE.md, y trabajé desde mi computador con una sesión SSH por terminal.



Verificación por capas (Parte A.4).

Paso	Capa	Comando	Resultado obtenido
1	Enlace/Red	ip -br addr (ambas VM)	enp0s8 UP con 192.168.56.10/24 (servidor) y 192.168.56.11/24 (cliente)
2	Red	ping -c 4 192.168.56.10	4/4 recibidos, 0 % de pérdida; rtt mín/media/máx = 0,354 / 0,556 / 0,752 ms
3	Transporte	nc -vz 192.168.56.10 5000 sin servidor	Connection refused
4	Transporte	nc -vz 192.168.56.10 5000 con servidor	succeeded!
5	Transporte	ss -ltnp en VM-SERVIDOR	LISTEN 0 1 0.0.0.0:5000 users:(("python3", pid=2408, fd=3)); Send-Q = 1 es el listen(1)
6	Aplicación	cliente_eco.py 192.168.56.10 5000	envié Hola e ingeniero, recibí HOLA e INGENIERO

```
Last login: Thu Oct 1 05:06:06 2026 from 192.168.56.1
santech@santechserver:~$ ip -br addr
lo                UNKNOWN      127.0.0.1/8 ::1/128
enp8s3            UP            10.0.2.15/24 metric 100 fd17:625c:f037:2:a08
:27ff:fe1d:a5ef/64 fe80::a00:27ff:fe1d:a5ef/64
enp8s8            UP            192.168.56.10/24 fe80::a00:27ff:fe02:388f/64
santech@santechserver:~$
```

A.1 ip -br addr, VM-SERVIDOR

```
Last login: Thu Oct 1 04:58:38 2026 from 192.168.56.1
santech@vm-cliente:~$ ip -br addr
lo                UNKNOWN      127.0.0.1/8 ::1/128
enp8s3            UP            10.0.2.15/24 metric 1024 fd17:625c:f037:2:a08
:27ff:fe9c:1c3d/64 fe80::a00:27ff:fe9c:1c3d/64
enp8s8            UP            192.168.56.11/24 fe80::a00:27ff:fe43:1c95/64
santech@vm-cliente:~$
```

A.1 ip -br addr, VM-CLIENTE

```
santech@vm-cliente:~$ ping -c 4 192.168.56.10
PING 192.168.56.10 (192.168.56.10) 56(84) bytes of data.
64 bytes from 192.168.56.10: icmp_seq=1 ttl=64 time=0.752 ms
64 bytes from 192.168.56.10: icmp_seq=2 ttl=64 time=0.571 ms
64 bytes from 192.168.56.10: icmp_seq=3 ttl=64 time=0.354 ms
64 bytes from 192.168.56.10: icmp_seq=4 ttl=64 time=0.550 ms
--- 192.168.56.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3106ms
rtt min/avg/max/mdev = 0.354/0.556/0.752/0.140 ms
santech@vm-cliente:~$
```

A.2 ping

```
santech@vm-cliente:~$ nc -vz 192.168.56.10 5000
nc: connect to 192.168.56.10 port 5000 (tcp) failed: Connection refused
santech@vm-cliente:~$
```

A.3 nc sin servidor

```
santech@vm-cliente:~$ nc -vz 192.168.56.10 5000
Connection to 192.168.56.10 5000 port [tcp/*] succeeded!
santech@vm-cliente:~$
```

A.4 nc con servidor

```
Last login: Thu Oct 1 16:26:22 2026 from 192.168.56.1
santech@santechserver:~$ ss -ltnp
State      Recv-Q    Send-Q      Local Address:Port      Peer Address:Port
LISTEN     0          4096              0.0.0.0:22              0.0.0.0:*
LISTEN     0          1                0.0.0.0:5000            0.0.0.0:*
users:((("python3",pid=2408,fd=3))
LISTEN     0          4096      127.0.0.0:53             0.0.0.0:*
LISTEN     0          4096      127.0.0.0:54:53         0.0.0.0:*
LISTEN     0          4096              [::]:22                  [::]:*
```

A.5 ss -ltnp

```
santech@vm-cliente:~$ cd ~/lab1/codigo/base
python3 cliente_eco.py 192.168.56.10 5000
[cliente] conectando a 192.168.56.10:5000 ...
[cliente] conectado
[cliente] extremo local : ('192.168.56.11', 45036) <- puerto efimero asignado por el S.O.
[cliente] extremo remoto : ('192.168.56.10', 5000)
[cliente] escriba un texto y presione Enter. Escriba 'salir' para terminar.
> Hola
[cliente] enviados 4 bytes: b'Hola'
[cliente] recibidos 4 bytes: b'HOLA'
> ingeniero
[cliente] enviados 9 bytes: b'ingeniero'
[cliente] recibidos 9 bytes: b'INGENIERO'
>
```

A.6 eco, cliente

```
[servidor] ---- conexion aceptada ----
[servidor] cliente remoto : 192.168.56.11:45036
[servidor] extremo local : ('192.168.56.10', 5000)
[servidor] extremo remoto : ('192.168.56.11', 45036)
[servidor] socket de escucha sigue siendo: ('0.0.0.0', 5000) (distinto objeto)
[servidor] recibidos 4 bytes: b'Hola'
[servidor] enviados 4 bytes: b'HOLA'
[servidor] recibidos 9 bytes: b'ingeniero'
[servidor] enviados 9 bytes: b'INGENIERO'
```

A.6 eco, servidor

Cuádrupla de la primera conexión: (192.168.56.11, 45036, 192.168.56.10, 5000), tomada de lo que imprimen `cliente_eco.py` y `servidor_eco.py`.

3. Desarrollo por experimento

Antes de ejecutar cada experimento anoté mi predicción. Los comandos exactos están en `experimentos/GUIA_E1_E6.md`. En las tablas, S1/S2 son terminales en VM-SERVIDOR y C1...C4 en VM-CLIENTE.

3.1 Tabla resumen: predicción / observación / explicación

Exp.	Mi predicción (antes de ejecutar)	Lo que observé	Explicación
E1	C2 y C3 quedarían esperando sin error, en una fila «como una pila», y recibirían su eco cuando C1 escribiera <code>salir</code> .	Sin error: <code>ss</code> mostró 3 filas ESTAB y la fila LISTEN con Recv-Q = 2. Al salir C1 se atendió a C2 y luego a C3: orden de llegada (FIFO), no pila. Un cuarto cliente no entró: <code>Connection timed out</code> a los 2 min 15 s.	Las conexiones solo difieren en el puerto efimero, y eso basta para que la cuádrupla sea única. El kernel completa el saludo y encola la conexión aunque la aplicación esté ocupada; con <code>listen(1)</code> caben 2 en la cola.
E2	El socket de la conexión tendría un puerto distinto al 5000 (confundí socket con puerto).	Escucha en <code>0.0.0.0:5000</code> ; conexión en <code>192.168.56.10:5000</code> ↔ <code>192.168.56.11:49114</code> . El puerto del servidor no cambió.	El socket de escucha está ligado a cualquier interfaz; el de la conexión pertenece a una cuádrupla concreta y tiene la IP por la que entró el SYN.
E3	Al cerrar con <code>salir</code> saldría un «exit» o un error; sin el <code>break</code> , un error; con cierre brusco, un bucle que se cuelga.	<code>recv()</code> devolvió 0 y el servidor siguió vivo. Sin <code>break</code> : bucle infinito de recibidos 0 bytes, 58,4 % de CPU. Apagado brusco: S1 no se enteró, ESTAB más de 12 min, servidor bloqueado en <code>recv()</code> .	<code>recv() = 0</code> solo significa que llegó un FIN; si no hay datos, <code>recv()</code> bloquea. Una VM apagada no envía FIN ni RST.
E4	<code>recv()</code> entregaría los tres mensajes de la ráfaga de una sola vez.	En 11 conexiones, tres agrupaciones distintas (3+3+4, 3+7 y 6+4). Con pausa 0,5 s el servidor recibió 3+3+4 y el cliente los 10 bytes juntos. El servidor cayó con <code>ConnectionResetError</code> y <code>BrokenPipeError</code> .	TCP entrega los bytes completos y en orden, pero no conserva las fronteras de cada <code>sendall()</code> ; la agrupación depende de la temporización.
E5	No registré predicción antes de ejecutarlo.	Sin <code>SO_REUSEADDR</code> : <code>OSError: [Errno 98] Address already in use</code> durante unos 60 s. Con <code>SO_REUSEADDR</code> : reinicio inmediato con un <code>TIME_WAIT</code> activo.	Quien cierra primero queda en <code>TIME_WAIT</code> ; sin la opción, el kernel no deja volver a hacer <code>bind()</code> a ese puerto.
E6	Todos los fallos serían instantáneos y con errores distintos.	F1, F2 y F4: <code>conexion rechazada</code> en 0,04 s. F3: <code>No route to host</code> en 3,15 s. F5: <code>Connection timed out</code> en 2 min 12 s.	Fallo rápido: el destino respondió (RST). Fallo lento: nadie respondió (ARP fallido o SYN descartado).

3.2 E1: la conexión tiene cuatro identificadores

Con S1 atendiendo a C1, y con C2 y C3 conectados, el servidor mostró:

<pre>\$ ss -tn state established '(sport = :5000)'</pre>	<pre>\$ ss -ltn '(sport = :5000)'</pre>
<pre>0 0 192.168.56.10:5000 192.168.56.11:58596 (C1)</pre>	<pre>State Recv-Q Send-Q Local Address:Port</pre>
<pre>0 0 192.168.56.10:5000 192.168.56.11:52992 (C3)</pre>	<pre>LISTEN 2 1 0.0.0.0:5000</pre>
<pre>0 0 192.168.56.10:5000 192.168.56.11:52990 (C2)</pre>	

Conexión	IP origen	Puerto origen	IP destino	Puerto destino
C1	192.168.56.11	58596	192.168.56.10	5000
C2	192.168.56.11	52990	192.168.56.10	5000
C3	192.168.56.11	52992	192.168.56.10	5000

- **E1.1** Las tres comparten la IP y el puerto del servidor y la IP del cliente; solo cambia el puerto efímero.
- **E1.2** Al llegar un segmento, el kernel busca la conexión por la cuádrupla completa; cada puerto efímero identifica un socket de datos distinto, con sus propios búferes, y la respuesta sale por ese socket.
- **E1.3** Desde un mismo cliente hacia el mismo IP:puerto, la cuádrupla solo varía en el puerto de origen. En VM-CLIENTE medí `ip_local_port_range = 32768 60999` (28 232 puertos) y `ulimit -n = 1024`: un solo proceso agota sus descriptores (unos 1020 sockets) mucho antes que los puertos.
- **E1.4** C2 y C3 no recibieron error: `connect()` terminó y enviaron `hola2` y `hola3`, pero el eco llegó a C2 al cerrarse C1, y a C3 al cerrarse C2. `listen(1)` fija el tamaño de la cola de conexiones ya establecidas, no cuántos clientes se atienden; el saludo lo completa el kernel y los datos esperan en el búfer del socket.
- **Predicción vs. resultado.** Acerté en la espera y fallé en la estructura: la cola de `accept()` es FIFO. Además, como el servidor es iterativo, C3 esperó también a C2.
- **Cuarto cliente.** Predije que C4 también entraría a la cola; no fue así. Con la cola llena (`Recv-Q = 2`, es decir `backlog + 1`), `ss` en el cliente mostró C4 en `SYN-SENT`, y terminó con `[Errno 110] Connection timed out` a los 2 min 15 s: el servidor descartó su SYN sin responder. Para el cliente, una cola llena se ve igual que un cortafuegos (F5). No lo verifiqué con `tcpdump`.

```
S1
santech@santechserver:~/lab1/codigo/hasse$ ss -tn state established 't sport
# :5000 '
Recv-Q Send-Q Local Address:Port Peer Address:Port
0 0 192.168.56.10:5000 192.168.56.11:58596
0 0 192.168.56.10:5000 192.168.56.11:52990
0 0 192.168.56.10:5000 192.168.56.11:52992
santech@santechserver:~/lab1/codigo/hasse$ ss -ltn
State Recv-Q Send-Q Local Address:Port Peer Address:Port
LISTEN 2 0 0.0.0.0:5000 0.0.0.0:*
santech@santechserver:~/lab1/codigo/hasse$

C2
santech@vm-cliente:~/lab1/codigo/hasse$ cd ~/lab1/codigo/hasse
santech@vm-cliente:~/lab1/codigo/hasse$ python3 cliente_echo.py 192.168.56.10
5000
[cliente] conectado a 192.168.56.10:5000 ...
[cliente] conectado
[cliente] extremo local : ('192.168.56.11', 52990) <- puerto efímero asign
ado por el S.O.
[cliente] extremo remoto : ('192.168.56.10', 5000)
[cliente] escriba un texto y presione Enter. Escriba 'salir' para terminar.
> hola2
[cliente] enviados 5 bytes: b'hola2'

C3
santech@vm-cliente:~/lab1/codigo/hasse$ cd ~/lab1/codigo/hasse
santech@vm-cliente:~/lab1/codigo/hasse$ python3 cliente_echo.py 192.168.56.10
5000
[cliente] conectado a 192.168.56.10:5000 ...
[cliente] conectado
[cliente] extremo local : ('192.168.56.11', 52992) <- puerto efímero asign
ado por el S.O.
[cliente] extremo remoto : ('192.168.56.10', 5000)
[cliente] escriba un texto y presione Enter. Escriba 'salir' para terminar.
> hola3
[cliente] enviados 5 bytes: b'hola3'
```

```
[servidor] ---- conexion aceptada ----
[servidor] cliente remoto : 192.168.56.11:52990
[servidor] extremo local : ('192.168.56.10', 5000)
[servidor] extremo remoto : ('192.168.56.11', 52990)
[servidor] socket de escucha sigue siendo: ('0.0.0.0', 5000) (distinto objet
o)
[servidor] recibidos 5 bytes: b'hola2'
[servidor] enviados 5 bytes: b'HOLA2'
[servidor] recv() devolvió 0 bytes -> el cliente cerró la conexión
[servidor] conexión cerrada. Vuelvo a accept()

[servidor] ---- conexion aceptada ----
[servidor] cliente remoto : 192.168.56.11:52992
[servidor] extremo local : ('192.168.56.10', 5000)
[servidor] extremo remoto : ('192.168.56.11', 52992)
[servidor] socket de escucha sigue siendo: ('0.0.0.0', 5000) (distinto objet
o)
[servidor] recibidos 5 bytes: b'hola3'
[servidor] enviados 5 bytes: b'HOLA3'
```

E1: S1, S2 (ss), C2 y C3

E1: S1 atiende a C2 y luego a C3

```
S1
santech@santechserver:~/lab1/codigo/hasse$ ss -ltn 't sport = :5000 '
State Recv-Q Send-Q Local Address:Port Peer Address:Port
LISTEN 2 0 0.0.0.0:5000 0.0.0.0:*
santech@santechserver:~/lab1/codigo/hasse$

C3
[cliente] extremo local : ('192.168.56.11', 36876) <- puerto efímero asignado por el S.O.
[cliente] extremo remoto : ('192.168.56.10', 5000)
[cliente] escriba un texto y presione Enter. Escriba 'salir' para terminar.
> hola3
[cliente] enviados 5 bytes: b'hola3'

C4
6.1
python3 cliente_echo.py 192.168.56.10 5000
[cliente] conectado a 192.168.56.10:5000 ...
[cliente] conectado
[cliente] extremo local : ('192.168.56.11', 51832) <- puerto efímero asignado por el S.O.
[cliente] extremo remoto : ('192.168.56.10', 5000)
[cliente] socket cerrado
real 2m15.487s
user 8m8.825s
sys 6m8.815s
santech@vm-cliente:~/lab1/codigo/hasse$

C1
[cliente] conectado
[cliente] extremo local : ('192.168.56.11', 36298) <- puerto efímero asignado po
r el S.O.
[cliente] extremo remoto : ('192.168.56.10', 5000)
[cliente] escriba un texto y presione Enter. Escriba 'salir' para terminar.
> hola
[cliente] enviados 4 bytes: b'hola'
[cliente] recibidos 4 bytes: b'HOLA'
>
```

E1: C4 en SYN-SENT y su timeout

3.3 E2: el servidor maneja dos sockets distintos

Socket	Dirección local	Puerto local	Extremo remoto
Escucha (LISTEN)	0.0.0.0	5000	0.0.0.0:*
Conexión (ESTAB)	192.168.56.10	5000	192.168.56.11:49114

- S1 imprimió `extremo local: ('192.168.56.10', 5000)` y `socket de escucha sigue siendo: ('0.0.0.0', 5000)` (distinto objeto), y `ss -tan` mostró las dos filas a la vez.
- **E2.1** El socket de escucha se ligó con `bind(("0.0.0.0", 5000))` y no tiene IP local concreta; el de la conexión sí, la IP por la que entró el SYN, porque forma parte de una cuádrupla.
 - **E2.2** Un socket en LISTEN no tiene flujo de datos: `recv()` sobre él falla con `osError (ENOTCONN)`. Esto lo tomo de la documentación; no lo probé.
 - **E2.3** Coincide con la cita de la guía: el socket de escucha nunca aparece en `recv()` ni `sendall()`; su único trabajo es producir un socket nuevo en cada `accept()`.
 - **Predicción vs. resultado.** Predije otro puerto y me equivoqué: un socket no es un puerto. Los dos usan el 5000; el puerto que cambia es el del cliente.


```
[cliente] escriba un texto y presione Enter. Escriba 'salir' para term
inar.

> HOLA
[cliente] enviados 4 bytes: b'HOLA'
[cliente] recibidos 4 bytes: HOLA
> ^c
[cliente] interrumpido por el usuario
[cliente] socket cerrado
santech@vm-cliente:~/lab1/codigo/base$ time python3 cliente_eco.py 192
.168.56.10 5000
[cliente] conectando a 192.168.56.10:5000 ...
[cliente] ERROR: conexion rechazada. Nadie escucha en 192.168.56.10:50
00
[cliente] socket cerrado

real    0m0.046s
user    0m0.033s
sys     0m0.012s
santech@vm-cliente:~/lab1/codigo/base$
```

F1: cliente

[illegible]

F4: solo 127.0.0.1

[illegible]

F5: ufw status verbose

[illegible]

F2: puerto 5001

```
x s2 x + v - □
```

```
>antexec@antexecserver:~/lab1/codigo/base$ date +%T; ss -tano state time-wait  
19:27:31  
Recv-Q Send-Q Local Address:Port Peer Address:Port  
192.168.56.10:5080 192.168.56.11:40994  
timer:(timewait,56sec,0)  
>antexec@antexecserver:~/lab1/codigo/base$ date +%T; ss -tano state time-wait  
19:31:54  
Recv-Q Send-Q Local Address:Port Peer Address:Port  
192.168.56.10:5080 192.168.56.11:38688  
timer:(timewait,55sec,0)  
>antexec@antexecserver:~/lab1/codigo/base$ ss -ltnp  
Recv-Q Send-Q Local Address:Port Peer Address:Port Process  
LISTEN 0 4096 127.0.0.0:0.22 0.0.0.0:*  
LISTEN 0 4096 127.0.0.23:0.0 0.0.0.0:*  
LISTEN 0 4096 127.0.0.34:0.33 0.0.0.0:*  
LISTEN 0 4096 [::]:22 [::]:*
```

```
>antexec@antexecserver:~/lab1/codigo/base$
```

```
x t1 x + v - □
```

```
>antexec@antexecserver:~/lab1/codigo/base$ python3 servidor_ecc.py  
{servidor} socket de escucha en ('0.0.0.0', 5080)  
{servidor} esperando conexiones... (CtrlC para terminar)
```

```
[servidor] --- conexion aceptada ---  
[servidor] cliente remoto : 192.168.56.11:38688  
[servidor] extremo local : ('192.168.56.10', 5080)  
[servidor] extremo remoto : ('0.0.0.0', 5880)  
[servidor] socket de escucha sigue funcionando: ('0.0.0.0', 5080) (distinto objeto)  
[servidor] recibidos n bytes: b'HOLA'  
[servidor] enviados d bytes: b'HOLA'  
<C  
[servidor] interrumpido por el usuario  
[servidor] socket de escucha cerrado  
>antexec@antexecserver:~/lab1/codigo/base$ python3 servidor_ecc.py  
[servidor] socket de escucha en ('0.0.0.0', 5880)  
[servidor] esperando conexiones... (CtrlC para terminar)
```

```
[servidor] --- conexion aceptada ---  
[servidor] cliente remoto : 192.168.56.11:38688  
[servidor] extremo local : ('192.168.56.10', 5080)  
[servidor] extremo remoto : ('0.0.0.0', 5880)  
[servidor] socket de escucha sigue funcionando: ('0.0.0.0', 5080) (distinto objeto)  
[servidor] recibidos n bytes: b'HOLA'  
[servidor] enviados d bytes: b'HOLA'  
<C  
[servidor] interrumpido por el usuario  
[servidor] socket de escucha cerrado  
>antexec@antexecserver:~/lab1/codigo/base$
```

F1: ss -ltnp en el servidor

```

$ nola
[cliente] enviados 4 bytes: b'hola'
[cliente] recibidos 4 bytes: b'OLA'
> time python3 cliente_eco.py 192.168.56.10 5000*c
[cliente] interrumpido por el usuario
[cliente] socket cerrado
santech@war-cliente:~/lab/codigo/base$ time python3 cliente_eco.py 192.168.56.1
0 5000
[cliente] conectando a 192.168.56.10:5000 ...
[cliente] ERROR de red: [Errno 110] Connection timed out
[cliente] socket cerrado

real    2m12.752s
user    0m0.041s
sys     0m0.006s
santech@war-cliente:~/lab/codigo/base$ |

```

F5: repetición

[illegible]

F3: No route to host, ip neigh FAILED

[illegible]

F5: Connection timed out

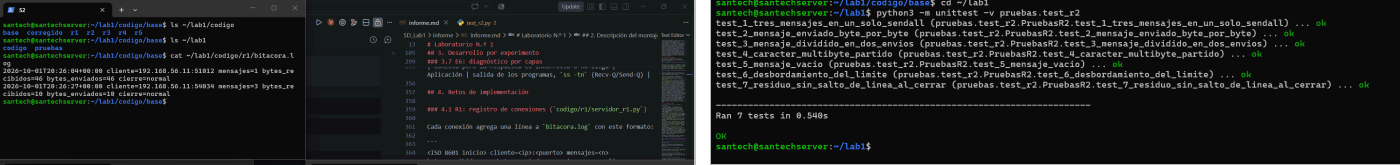
4. Retos de implementación

4.1 R1: registro de conexiones (codigo/r1/servidor_r1.py)

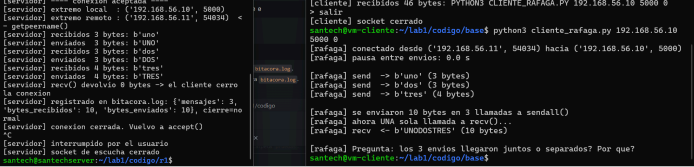
del límite y residuo al cerrar. Para comprobar que las pruebas detectan errores reales, las ejecuté contra copias del servidor con fallos introducidos a propósito:

Fallo introducido	Prueba que lo detecta
if en lugar de while	test_1
Búfer reiniciado en cada recv()	test_2, test_3 y test_4
Decodificar cada recv()	test_4

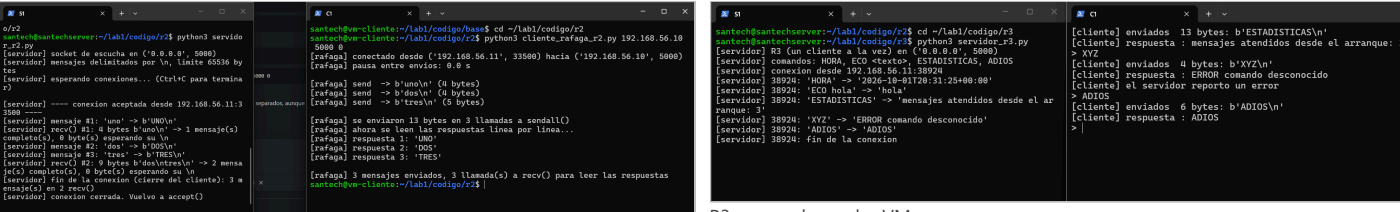
Con la misma ráfaga de E4 (cliente_rafaga_r2.py ... 0) en las VM, el servidor hizo **2 recv()** (uno\n y luego dos\ntres\n) y aun así identificó **3 mensajes**; el cliente recibió UNO, DOS y TRES por separado, y el servidor no se cayó. TCP agrupa igual que antes, pero ahora el delimitador permite reconstruir los mensajes.



R2: 7 pruebas OK en VM-SERVIDOR



R1: bitacora.log en VM-SERVIDOR



R2: 3 mensajes en 2 recv()

R3: comandos en las VM

4.3 Retos adicionales (R3, R4, R5) y versión corregida

R3: comandos (codigo/r3/servidor_r3.py), sobre el protocolo de líneas de R2. Resultados en las VM:

Comando	Respuesta del servidor
HORA	2026-10-01T20:31:25+00:00
ECO hola	hola
ESTADISTICAS	mensajes atendidos desde el arranque: 3 (cuenta HORA, ECO y la propia consulta)
XYZ	ERROR comando desconocido; la sesión sigue abierta
ADIOS	ADIOS y el servidor cierra la conexión

R4: varios clientes (codigo/r4/servidor_r4.py). El hilo principal solo ejecuta accept() y cada conexión se atiende en su propio threading.Thread. En las VM dejé C1 conectado sin escribir, y C2 y C3 recibieron la respuesta a HORA de inmediato (20:35:09 y 20:35:14); el servidor llegó a 4 hilos de clientes. Predije que C2 respondería «porque C1 no tiene ninguna solicitud»; acerté el resultado, pero la causa es otra: en E1 C1 también estaba callado y C2 esperó. Lo que cambia es que ya no hay un único hilo bloqueado en el recv() de C1. La comparación con R3 no la repetí en las VM; en mi prueba previa, con A conectado 2 s, B y C recibieron su respuesta a los 2,01 s con R3 y a los 0,20 s con R4.

El contador de ESTADISTICAS es compartido: valor += 1 es leer, sumar y escribir, y dos hilos pueden pisarse (condición de carrera). Lo protegí con un threading.Lock (ContadorSeguro). Resultado de demo_carrera.py en VM-SERVIDOR (8 hilos x 2000 incrementos):

Caso	Esperado	Obtenido	Perdidos	Tiempo
1. Sin Lock (valor += 1)	16000	16000	0	0,01 s
2. Sin Lock, ventana leer/escribir ampliada con time.sleep(0)	16000	2002	13998	0,21 s
3. Con Lock, misma ventana	16000	16000	0	1,57 s

Predije que se perderían incrementos; solo ocurrió al ampliar la ventana. Lo importante es la fila 1: sin Lock **la prueba pasa**, porque el cambio de hilo justo entre leer y escribir es raro; por eso este error sobrevive a las pruebas y aparece bajo carga. El Lock evita las pérdidas a cambio de tiempo.

R5: RTT (codigo/r5/cliente_rtt.py, contra servidor_r2.py). Medí cada mensaje con time.perf_counter() del cliente, desde antes de sendall() hasta tener la respuesta completa, sobre una sola conexión para que el saludo no entre en la medición.

- 6. ¿Por qué `accept()` devuelve un socket nuevo?** Porque el de escucha debe seguir en LISTEN para los próximos clientes, y cada conexión necesita su cuádrupla, su estado y sus búferes. En E2 `ss` mostró las dos filas a la vez y el servidor reportó objetos distintos.
- 7. «TCP no conserva las fronteras», con datos de E4.** Tres `sendall()` produjeron en el servidor dos o tres `recv()` según la corrida (3+3+4, 3+7, 6+4), y con pausa 0,5 s el cliente recibió los tres ecos en un solo `recv()`. Siempre llegaron 10 bytes en orden.
- 8. Tres cosas que TCP no garantiza.** (a) Que cada `send()` llegue en un `recv()` separado (E4). (b) Que el otro extremo se entere pronto de una caída: en E3.4 la conexión siguió en ESTAB más de 12 minutos. (c) Cuando llegan los datos: no hay límite de latencia, y un ACK solo dice que el kernel remoto recibió los bytes, no que la aplicación los procesó.
- 9. ¿Por qué `sendall()` es preferible a `send()`?** `send()` puede aceptar solo una parte de los bytes y deja el resto al programador; `sendall()` repite `send()` hasta entregar todo o lanza excepción, aunque entonces no se sabe cuántos bytes salieron. En R5 cada mensaje de 8000 B salió con un solo `sendall()` y el servidor lo leyó en varios `recv()` de 1024 B.
- 10. Si TCP es fiable, ¿por qué delimitar mensajes?** Porque fiabilidad y fronteras son cosas distintas: TCP garantiza qué bytes llegan y en qué orden, no dónde termina cada mensaje. Esa información solo existe en la aplicación. R2 funciona porque ninguna capa inferior lo hacía: 2 `recv()`, 3 mensajes.
- 11. REST/gRPC: ¿desaparecen esas llamadas?** No, quedan ocultas: la biblioteca cliente ejecuta `socket()`, `connect()`, `send()` y `recv()`, y el kernel sigue haciendo el saludo. HTTP/2 y gRPC resuelven el problema de E4 con marcos que indican su longitud. Esto lo razono desde la documentación; no lo comprobé con `strace`.
- 12. Tres problemas que este laboratorio no enfrenta.** (a) Fallos parciales: un nodo cae y el otro no lo sabe (E3.4 es un anticipo); hacen falta timeouts y detección de fallos. (b) Ausencia de reloj global: me pasó al comparar mi hora local (UTC-5) con la de las VM (UTC) en E3.4. (c) Serialización de datos estructurados: aquí solo viaja texto; enviar estructuras exige acordar un formato (JSON, Protobuf) y el orden de bytes. También quedan fuera la seguridad (todo viaja en claro) y la concurrencia en los servidores iterativos.
- 13. Acuerdos escritos e implícitos.** Escritos: la dirección y el puerto (`HOST`, `PUERTO` y argumentos) y la codificación UTF-8, pero escrita dos veces, una por programa, sin contrato común. Implícitos: el significado del mensaje (eco en mayúsculas, dónde termina, qué es un error). El riesgo es que si un extremo cambia, el otro falla en silencio. Ejemplo real: `servidor_eco.py` aplica `.upper()` sobre bytes, y en mi prueba en Windows «mañana» volvió como `MAÑANA`.
- 14. Si cambiara TCP por UDP.** Dejaría de haber conexión y saludo, garantía de entrega y orden, y control de flujo y congestión. UDP sí conserva fronteras (cada `sendto()` es un datagrama), así que lo de E4 desaparecería, y `recv() = 0` ya no significaría «cerró». Tendría que implementar yo números de secuencia, confirmaciones con retransmisión, detección de duplicados y un límite de tamaño por mensaje.

6. Dificultades encontradas y cómo se resolvieron

- SSH no estaba disponible al principio.** `systemctl status ssh` respondió `Unit ssh.service could not be found` y luego mostró el servicio `inactive (dead)`, activado solo por `ssh.socket`. Lo resolví siguiendo el paso a paso de `red/DESPLIEGUE.md`.
- Netplan y el clon.** El archivo de las VM era `/etc/netplan/60-hostonly.yaml` y no el nombre del documento de despliegue, y al guardarlo en nano escribí mal el nombre (`60-hostonly.yaml1rr`). VM-CLIENTE, por ser un clon, conservaba el nombre `santechserver` y la IP .10: escribí `hostnamectl set-hostname -vm-cliente` con un guion de más (`invalid option -- 'v'`) y usé una ruta equivocada en `sed`, antes de lograr `vm-cliente` y `192.168.56.11/24`. Lo corregí siguiendo el paso a paso y lo comprobé con `ip -br addr`.
- Relojes distintos.** Las VM usan UTC y mi computador UTC-5; para medir E3.4 tuve que convertir la hora del apagado.
- Errores al ejecutar.** Corrí `cliente_eco.py` sin `python3` y desde la carpeta equivocada, intenté ejecutar `servidor_e3.py` antes de crearlo con `sed`, y en E1 pegué un comando `ss` dentro del cliente, que se envió como mensaje. Lo resolví trabajando desde `~/lab1/codigo/base` y repitiendo cada paso en la terminal correcta.
- El servidor base se cayó en E4** con `ConnectionResetError` y `BrokenPipeError`, y los clientes siguientes recibieron `Connection refused`. Lo reinicié para continuar, y en R1 y R2 capturé esas excepciones para que el servidor no muera.
- Resultados que cambian con la misma orden** (E4): tuve que registrar muchas corridas para poder afirmar algo.
- Perdí la salida de dos corridas de E4** al limpiar la pantalla del cliente; tuve que repetirlas una por una con captura. Aprendí a capturar antes de limpiar.
- No registré mi predicción de E5** antes de ejecutarlo; lo dejé escrito así en vez de reconstruirla.
- Esperas largas y riesgo con el cortafuegos.** F5 y el cuarto cliente de E1 tardaron más de 2 minutos; antes de activar `ufw` permití el puerto 22 para no perder SSH, y al final verifiqué `Status: inactive`.
- Diferencias de mi prueba previa en Windows:** `python3` abre la Microsoft Store (usé `python`), la bitácora salía con CRLF (fijé `newline="\n"`) y `bytes.upper()` no convierte la ñ (en R2 decodifico antes y uso `str.upper()`).

7. Conclusiones

1. El servidor usa dos tipos de socket, uno que escucha y uno por conexión, y los dos con el puerto 5000: un socket no es un puerto. El kernel distingue las conexiones por la cuádrupla, y entre clientes de la misma VM solo cambia el puerto efímero.
2. Un servidor iterativo deja a los demás clientes en la cola de `accept()`, que es FIFO y limitada (`backlog + 1`): el cuarto cliente terminó en timeout. R4, con un hilo por cliente, eliminó la espera, y su contador compartido necesitó un `lock` (13 998 de 16 000 incrementos perdidos sin él al ampliar la ventana).
3. TCP es un flujo de bytes fiable y ordenado sin fronteras: el mismo código dio tres agrupaciones distintas. Todo protocolo sobre TCP debe delimitar sus mensajes; R2 lo hizo con `\n`, búfer persistente y extracción con `while`, pasó sus 7 pruebas y no se cayó.
4. Mucho ocurre en el kernel y no en mi código: el saludo, las colas, `TIME_WAIT` (unos 60 s sin `SO_REUSEADDR`), los RST que tumbaron el servidor base y la conexión medio abierta de más de 12 minutos tras apagar el cliente.
5. Para diagnosticar conviene subir por capas y mirar cuánto tarda el error: rápido significa que alguien respondió; lento, silencio. Tres fallos distintos dieron el mismo error en 0,04 s y solo `ss -ltnp` en el servidor los separó.
6. Varias de mis predicciones fallaron (pila, puerto distinto, errores distintos, todo instantáneo): no basta con razonar, hay que medir, y una prueba que pasa una vez no demuestra que el programa esté bien.
7. REST, gRPC o las colas de mensajes siguen apoyándose en estas llamadas, y además deben resolver lo que este laboratorio no cubre: fallos parciales, tiempo y serialización.

8. Referencias

- Coulouris, G., Dollimore, J., Kindberg, T., y Blair, G. (2012). *Distributed systems: Concepts and design* (5.^a ed.). Addison-Wesley.
- Eddy, W. (Ed.). (2022). *RFC 9293: Transmission Control Protocol (TCP)*. Internet Engineering Task Force. <https://www.rfc-editor.org/info/rfc9293/>
- Kurose, J. F., y Ross, K. W. (2021). *Computer networking: A top-down approach* (8.^a ed.). Pearson.
- Oracle Corporation. (s.f.). *Oracle VirtualBox user manual — Chapter 6: Virtual networking*. <https://www.virtualbox.org/manual/ch06.html>
- Python Software Foundation. (s.f.). *Socket programming HOWTO*. <https://docs.python.org/3/howto/sockets.html>